

IEEE SSCS Egypt Chapter
2026 Student Design Competition

FPGA-Based Edge-AI Vision Accelerator

Streaming 3 x 3 CNN Convolution Engine on a Single Multi-Pumped DSP

Final Design Report | Target: PYNQ-Z2 (Zynq-7000 xc7z020clg400-1)

Team Members

No.	Name
1	Omar Wael
2	Ziad Elkhiaat
3	Salwa Galal

1. Executive Summary.....	3
Key results.....	3
2. Specification Compliance and Results Table.....	3
3. Architecture Overview	5
3.1 Top-level interface	6
4. Datapath	7
4.1 Line buffer and window generation (M1, M2)	7
4.2 MAC engine (M3)	7
4.3 Output handling (M5)	9
4.4 Output FIFO (M7)	9
5. Control FSM (M6).....	10
6. Memory Organisation	11
7. Clocking and Timing Constraints	11
8. Fixed-Point Bit-Width Analysis.....	11
9. RTL Implementation Details.....	12
9.1 Implementation techniques.....	13
10. Verification.....	13
10.1 Golden reference model	13
10.2 Testbenches	13
10.3 Test images	14
10.4 Results.....	14
11. FPGA Implementation Results	18
11.1 Utilization.....	18
11.2 Timing.....	19
11.3 Power	19
11.4 Placement	20
11.5 Figure of Merit	21
12. Design Trade-offs and Optimisation History.....	21
12.1 Decisions and alternatives	21
13. Bonus Features	22
14. Assumptions.....	22
15. Conclusion.....	22
Appendix: reproducing the results	22

1. Executive Summary

This report describes a streaming 3 x 3 convolution accelerator for grayscale images, written in Verilog-2001 and implemented on the Zynq-7000 xc7z020 device of the PYNQ-Z2 board. The accelerator accepts one 8-bit pixel per system clock cycle in raster order and, once its pipeline and output buffer are filled, delivers one 16-bit output pixel per system cycle without gaps. Kernel coefficients are programmable 8-bit signed fixed-point values held in two independently writable banks, and an optional ReLU can be applied to every output.

The design is shaped by the competition Figure of Merit, $FOM = \text{Throughput} / (\text{Power} \times (\text{LUTs} + 50 \times \text{DSPs} + 100 \times \text{BRAMs}))$. DSP slices are weighted 50 times more than a LUT, BRAMs 100 times, and flip-flops are not counted at all. The architecture therefore computes all nine multiplications of each output on a **single DSP48E1** that runs at six times the system clock and packs two products into each of its operations. It keeps every buffer either in flip-flops or in small LUT-RAMs, and replaces binary counters and fabric adders with LFSRs, shift registers and DSP-internal logic wherever possible.

Key results

- **Resources:** 226 LUTs (194 logic + 32 LUT-RAM), 1233 flip-flops, 1 DSP48E1, 0 BRAM.
- **Throughput:** 1 output pixel per system cycle (gapless; verified by test IMG-TC4).
- **Maximum frequency:** DSP clock 300 MHz met (WNS +0.055 ns), giving a 50 MHz system clock, i.e. 50 Mpixel/s.
- **Power:** 0.112 W total on-chip at the 100 MHz reference clock (SAIF-based, confidence High).
- **FOM:** $1 / (0.112 \times (226 + 50 \times 1)) = 0.0324$ at the 100 MHz reference clock; 0.0284 at the 300 MHz clock.
- **Verification:** 16 image-based testbenches, compared bit-for-bit with a Python golden model.
- **Bonus features:** one-output-pixel-per-cycle pipelined architecture, multiple kernels (2 banks), ReLU, Sobel edge-detection demo.

2. Specification Compliance and Results Table

Table 1 follows the format required by the competition announcement. "Specification" is the requirement; "Team Result" is what this design achieves.

Table 1. Required results table.

Parameter	Specification	Team Result	Units	Comments
Input image size	$\geq 32 \times 32$	32 x 32 (parameter IMG_WIDTH)	pixels	Square image; any IMG_WIDTH ≥ 6 by parameter
Input precision	Fixed-point unsigned	UQ8.0 (8-bit unsigned)	bits	Grayscale 0-255, matches image sensors
Kernel precision	8-bit signed	Q3.4 (8-bit signed)	bits	Range -8.0 to +7.9375, step 1/16
Architecture type	N x N CNN convolution	Streaming line buffer + transposed-form 3 x 3 MAC on one 6x multi-pumped DSP	-	Stride 1, valid convolution (30 x 30 outputs)
Multipliers / MACs	-	1 DSP48E1; 9 multiplications per output as 6 DSP operations per system cycle	DSP	Two products packed per operation

Parameter	Specification	Team Result	Units	Comments
Pipeline stages	-	M3: 6, M5: 2, FIFO: 1	system cycles	DSP: 3 registers at the 6x clock
Latency	-	8 (window to result); ~135 (first pixel to first output)	system cycles	Includes line-buffer fill and 58-result FIFO reserve
Throughput	-	1	output pixels / cycle	Gapless output stream (IMG-TC4)
FPGA utilization	-	226 LUT, 1233 FF, 1 DSP, 0 BRAM	-	LUTs: 194 logic + 32 LUT-RAM
Maximum frequency	-	300 (DSP clock) / 50 (system clock)	MHz	WNS +0.055 ns at 3.333 ns
Power estimate	-	0.112 (100 MHz clk); 0.126 (300 MHz clk)	W	Total on-chip, SAIF, confidence High
Verification status	Golden-model comparison	16 image tests, bit-exact vs Python model	-	See Section 10
FOM	Report	0.0324 (100 MHz clk); 0.0284 (300 MHz clk)	1 / (W x LUT-eq.)	Throughput = 1

3. Architecture Overview

Figure 1 shows the top-level structure. Pixels enter M1, which delays the stream by one and two image rows so that three vertically aligned pixels (the newest column of the 3 x 3 window) are available every cycle. M2 decides when a column completes a valid window. M3 multiplies each new column by the kernel and accumulates the partial sums of the three window columns over time. M5 rounds and applies ReLU, and M7 buffers results so the output stream has no gaps. M4 stores the two kernel banks, M6 sequences each pass, and a small clock generator provides the 6x DSP clock and the system clock.

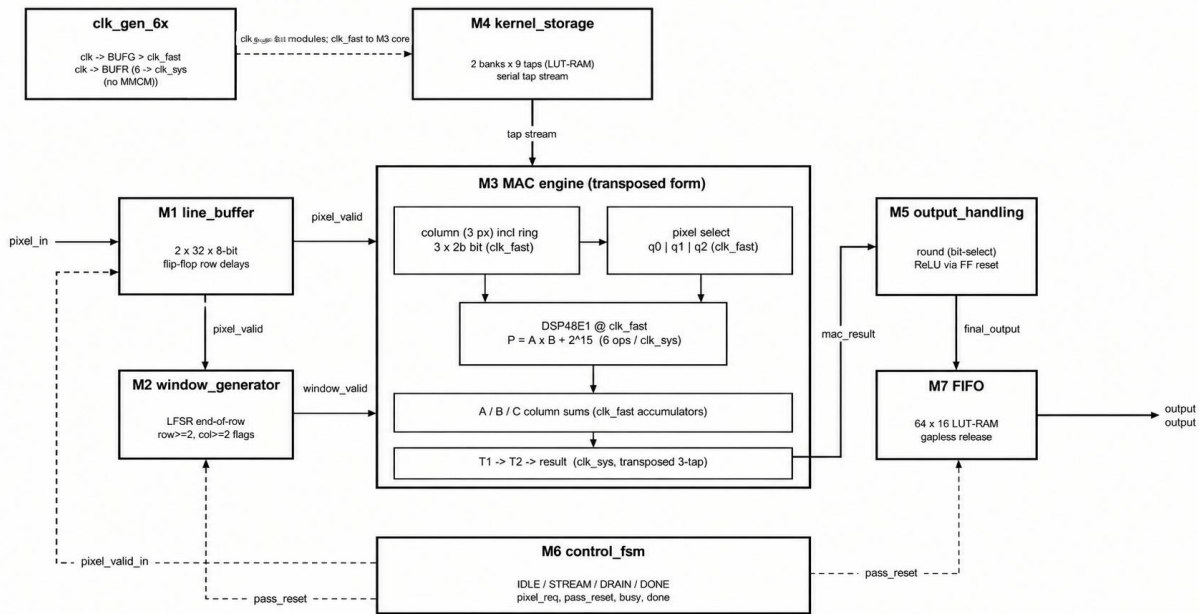


Figure 1. Top-level block diagram (solid: data, dashed: control).

Table 2. Modules and their roles.

Module	File	Role
M1	line_buffer.v	Two IMG_WIDTH-deep row delays (flip-flops); outputs the newest column of three pixels.
M2	window_generator.v	Position tracking; asserts window_valid when the column completes a 3 x 3 window.
M3	mac.v	Transposed-form MAC on one DSP48E1 at 6x clock with packed lanes; accumulators; rounding bias.
M4	kernel_storage.v	Two 9-tap banks in one LUT-RAM; streams the selected bank to M3.
M5	output_handling.v	Round-half-up, overflow handling, optional ReLU.
M6	control_fsm.v	Pass sequencing: IDLE, STREAM, DRAIN, DONE; pixel handshake.
M7	FIFO.v	64 x 16 output buffer; releases a gapless stream after a fixed reserve.
-	clk_gen_6x.v	BUFG (DSP clock) and BUFR divide-by-6 (system clock); no MMCM.
-	lfsr_counter.v	LFSR event counter used by M2, M6 and M7.
-	cnn_accelerator_top.v	Top level: connects all modules.

3.1 Top-level interface

Table 3. *Top-level ports (all synchronous to the system clock except clk).*

Port	Dir.	Width	Description
clk	in	1	6x reference clock (DSP clock); the system clock is clk / 6
rst	in	1	Synchronous reset (control state only)
start	in	1	Pulse to begin a pass
busy / done	out	1 / 1	Pass in progress / pass finished and all results released
kernel_wr_en, _bank, _addr, _data	in	1, 1, 4, 8	Write one tap (0-8) of one bank per cycle; taps 9-15 are ignored
kernel_select	in	1	Bank used by the next pass
relu_enable	in	1	Apply ReLU to every output
pixel_in / pixel_in_valid	in	8 / 1	Raster-order pixel stream
pixel_req	out	1	Accelerator is accepting pixels (STREAM state)
output_pixel / output_valid	out	16 / 1	Result stream (Q15.0 signed)
frame_done	out	1	Last result of the pass has been released

4. Datapath

4.1 Line buffer and window generation (M1, M2)

M1 holds two shift registers of IMG_WIDTH pixels each. On every accepted pixel both shift by one; the input pixel, the output of the first register (one row earlier) and the output of the second register (two rows earlier) form the newest window column (q_0, q_1, q_2 = rows $R-2, R-1, R$). The registers shift only when a pixel is accepted, so a stalled source simply freezes the pipeline. Their contents are never reset: M2 only raises `window_valid` after two full rows and two columns of real data have entered, so no power-up value can be observed.

Because M3 works in transposed form (Section 4.2), no 3×3 window register is needed: M2 only has to decide when the current column completes a valid window. It tracks three facts: the end of each row (an LFSR that flags the IMG_WIDTH-th pixel), "column ≥ 2 " and "row ≥ 2 " (two 2-bit shift registers of flags). `window_valid` is their AND with the pixel-accepted signal. At each row boundary this produces a gap of `KERNEL_SIZE - 1 = 2` cycles without a valid window, which M7 later absorbs.

4.2 MAC engine (M3)

Transposed-form accumulation

Let $q_r(x)$ be the pixel of window row r in image column x , and w_{rc} the kernel weight in row r , column c . The output for the window ending at column x is

$$y(x) = \sum_r [w_{r0} q_r(x-2) + w_{r1} q_r(x-1) + w_{r2} q_r(x)]$$

Each new pixel therefore meets all three weights of its kernel row over three consecutive windows. Instead of multiplying the nine window taps every cycle, M3 multiplies every new pixel once by its three row weights and forwards the column partial sums:

$$A = \sum_r w_{r0} q_r, \quad B = \sum_r w_{r1} q_r, \quad C = \sum_r w_{r2} q_r$$

$$T1 \leftarrow A, \quad T2 \leftarrow T1 + B, \quad y \leftarrow T2 + C \quad \Rightarrow \quad y(x) = A(x-2) + B(x-1) + C(x)$$

$T1, T2$ and y update only when a pixel is accepted, so stalls are handled exactly like M1. At a row boundary the first two columns of the new row refill $T1$ and $T2$ before `window_valid` rises again.

Two products per DSP operation

A pixel q is multiplied by two weights at once by packing both weights into the 25-bit A port of the DSP48E1 (Figure 2): $A = w_0 \times 2^{16} + w_1$, which fits in 25 signed bits for any 8-bit weights. The DSP post-adder adds a constant 2^{15} :

$$P = q \times A + 2^{15} = (q \times w_0) \times 2^{16} + (q \times w_1 + 2^{15})$$

Since $q \times w_1$ lies in $[-32640, 32385]$, the low lane $q \times w_1 + 2^{15}$ lies in $[128, 65153]$: it can neither borrow from nor carry into bit 16. Both products are therefore recovered with no correction logic: $q \times w_0 = P[31:16]$, and $q \times w_1 = P[15:0] - 2^{15}$, which is simply $P[15:0]$ with its top bit inverted. Single-weight operations use a sign-extended weight and read the low lane in the same way.

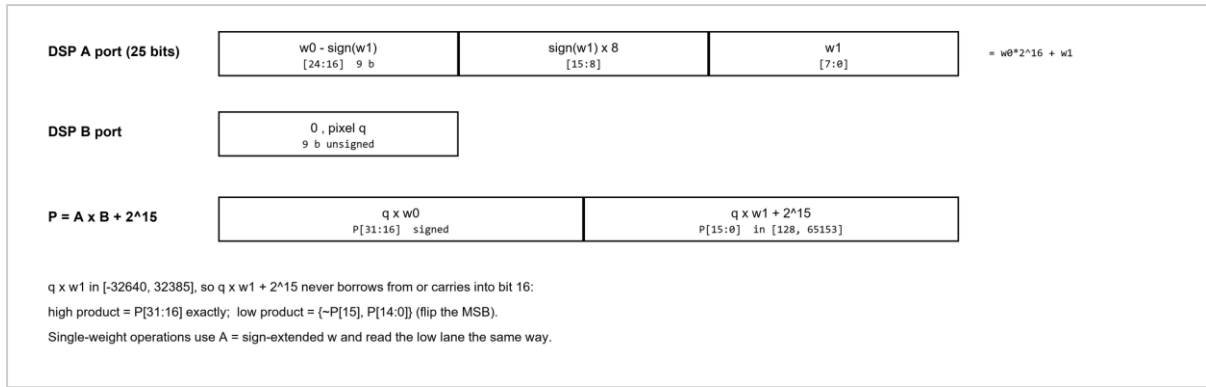


Figure 2. Operand packing on the DSP48E1 A port and lane extraction from P.

6x multi-pumping on one DSP

Per system cycle the kernel needs 9 products, which the packing reduces to 6 DSP operations: for each of the three rows, one packed operation ($q_r \times (w_{r0}, w_{r1})$) and one single operation ($q_r \times w_{r2}$). A single DSP48E1 executes these six operations in the six cycles of a clock running at six times the system clock (Figure 3). The DSP keeps its full A/B, M and P register pipeline, so it runs at 300 MHz on the -1 speed grade.

- **Operand ring.** The six A operands are held in a ring of six 25-bit registers on the fast clock that rotates once per system cycle, so the operand needed in each phase is always at the head of the ring. This replaces a 25-bit 6:1 multiplexer with flip-flops, which the FOM does not count.
- **Pixel select.** The B operand is one of the three column pixels (a 3:1 select), each used by two consecutive operations.
- **Phase tracking.** A bit that toggles on every system clock edge is sampled on the fast clock; its change pins a 3-bit phase counter, so the design never assumes a particular phase relationship between the two clocks.
- **Accumulation on the fast clock.** DSP results appear one per fast cycle. The high and low lanes of the packed operations are added into running sums A and B, and the single operations into C. The sums are copied to a holding register once complete and read at the next system clock edge; no fabric adder tree is needed.
- **Rounding bias.** The C sum starts from $2^{\text{FRAC_BITS}-1} = 8$ instead of 0, so the result leaving M3 already contains the rounding constant used by M5.

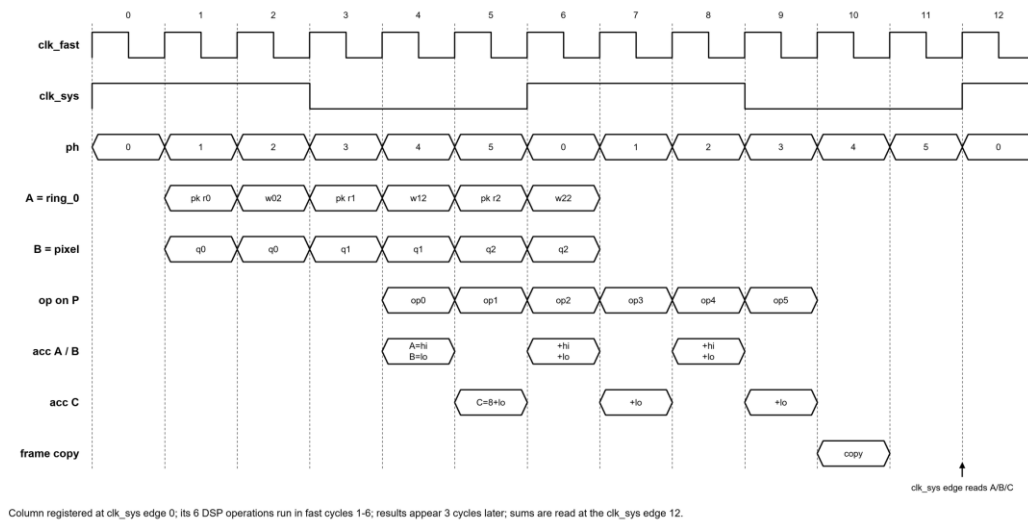


Figure 3. Schedule of the six DSP operations within one system clock period.

Weight delivery

M4 streams the selected bank one tap per cycle (row-major). M3 builds each packed operand when a row's column-1 tap arrives, using the column-0 tap received just before it, with one shared 9-bit subtractor (the high lane is $w_0 - \text{sign}(w_1)$ because the sign extension of w_1 contributes -2^{16} when w_1 is negative). The finished operand is inserted into the ring in the phase in which its slot passes the ring input, without disturbing the other five slots. The stream runs for one 10-tap sweep after every kernel write or bank change and then stops, so the operand path does not switch during a pass.

4.3 Output handling (M5)

- **Rounding:** round half up. Because M3 already added $2^{\text{FRAC_BITS}-1}$, rounding is only the removal of the $\text{FRAC_BITS} = 4$ fractional bits (a bit-select, no adder).
- **Overflow:** the largest possible magnitude of the 3×3 sum is $9 \times 255 \times 128 = 293760$, i.e. 18360 after the shift, well inside the 16-bit signed output range (32767). Overflow is therefore impossible by construction. The top level computes this bound from the parameters and builds a saturation stage only if a different choice of widths could overflow.
- **ReLU:** when `relu_enable` is set, a negative result becomes 0. This is implemented as the synchronous reset of the output register, which uses the flip-flops' own reset pins instead of a 16-bit multiplexer.
- M5 has two pipeline registers, so its latency is two system cycles.

4.4 Output FIFO (M7)

Valid windows occur in bursts of $\text{IMG_WIDTH} - 2$ per row, separated by 2-cycle gaps, while the input runs at one pixel per cycle. M7 absorbs these gaps: it stores results in a 64×16 LUT-RAM and starts releasing only after a reserve of $\text{RELEASE_THRESHOLD} = 2 \times \text{IMG_WIDTH} - 6 = 58$ results has been built. From then on it releases one result per cycle, and the reserve never runs out before the frame ends, giving a gapless output stream. The peak occupancy is 57 words, so 64 entries suffice.

Before release nothing is read, so the number of stored words equals the write pointer; the release point is simply $\text{wr_ptr} = \text{RELEASE_THRESHOLD} - 1$ with a write in progress. After release, "not empty" is $\text{wr_ptr} \neq \text{rd_ptr}$. An LFSR counts released words and raises `frame_done` after the last one.

5. Control FSM (M6)

M6 is a four-state Moore machine with a synchronous reset (Figure 4). It sequences one pass over the image and implements the pixel handshake: the source presents a pixel with pixel_in_valid whenever pixel_req is high, and a pixel is accepted in cycles where both are high.

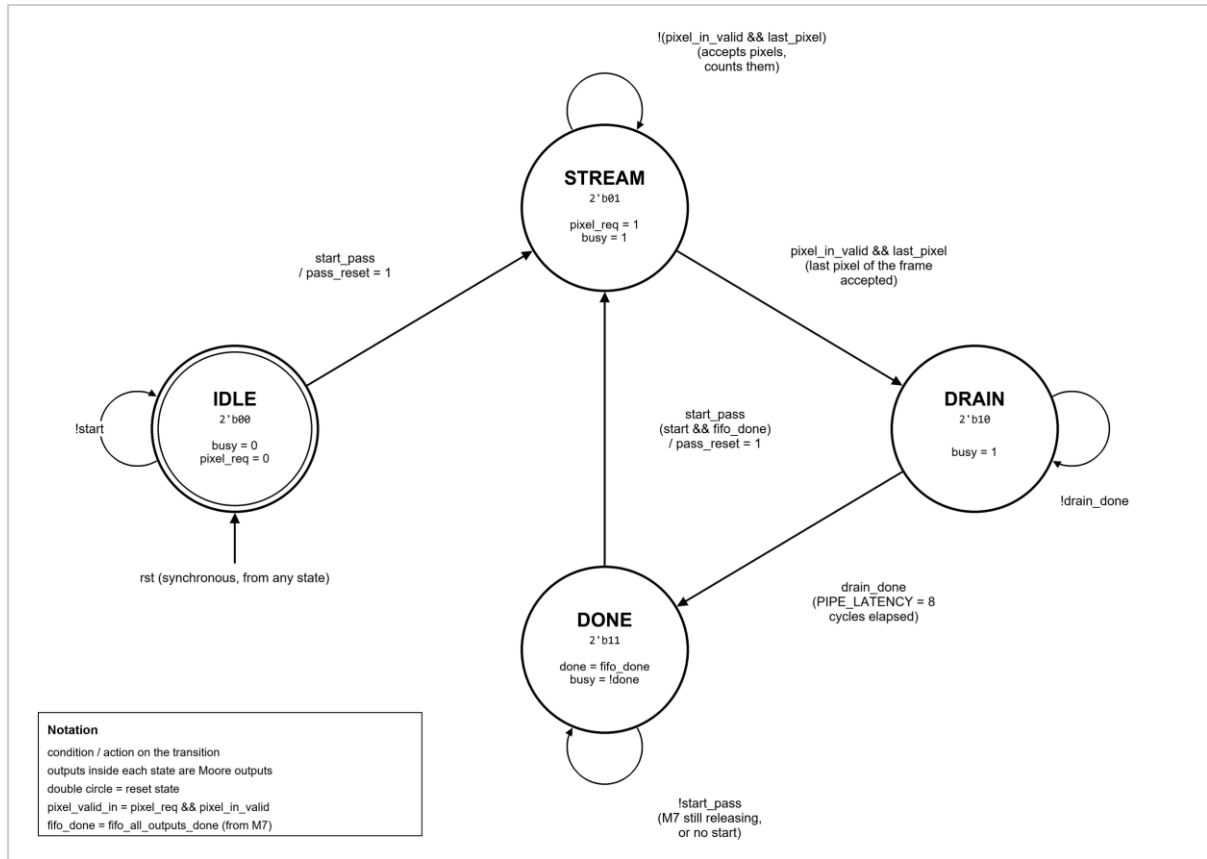


Figure 4. State diagram of the control FSM (M6).

Table 4. States, outputs and transitions.

State	Code	Outputs	Leaves when	Next state
IDLE	00	busy = 0, pixel_req = 0	start (pass_reset pulse)	STREAM
STREAM	01	pixel_req = 1, busy = 1	last pixel of the frame accepted	DRAIN
DRAIN	10	busy = 1	PIPE_LATENCY = 8 cycles elapsed	DONE
DONE	11	done = fifo_all_outputs_done, busy = !done	start && fifo_all_outputs_done (pass_reset pulse)	STREAM

- **Back-to-back passes.** DONE goes directly to STREAM on start, so a second pass (for example with the other kernel bank) needs a single start pulse. It is only accepted once M7 has released every result of the previous pass.
- **pass_reset.** A one-cycle pulse on every transition into STREAM clears the position tracking in M2 and the FIFO pointers in M7, so a new pass never inherits state from the previous one.
- **Stall-safe counting.** STREAM counts accepted pixels, not cycles, so a source that pauses cannot end the pass early. The counter is an LFSR whose terminal state is computed at elaboration.

- **Drain timer.** DRAIN lasts $\text{PIPE_LATENCY} = 8$ cycles ($M3\ 6 + M5\ 2$), measured with a 7-bit shift register instead of a binary counter.
- **Reset.** rst returns the machine to IDLE from any state; the design goes idle with no output (verified by IMG-TC15).

6. Memory Organisation

The FOM counts LUTs, DSPs and BRAMs but not flip-flops. Each storage element was therefore placed where it costs the fewest counted resources (Table 5). No BRAM is used: one BRAM tile would cost 100 LUT-equivalents, more than this design's entire M3 logic.

Table 5. Storage in the design.

Storage	Module	Size	Implementation	Reason
Row delays (line buffer)	M1	2 x 32 x 8 bits	512 flip-flops	0 LUTs; SRLs would cost ~16 LUTs
Kernel banks	M4	2 x 9 x 8 bits (32 x 8 RAM)	8 LUT-RAMs	Cheapest readable store; flip-flops would need an 18:1 read mux
DSP operand ring	M3	6 x 25 bits	150 flip-flops	Removes a 25-bit 6:1 mux
Column sums, T1, T2	M3	18-20 bits each	Flip-flops	Pipeline state
Output FIFO	M7	64 x 16 bits	24 LUT-RAMs	One LUT stores 64 bits; flip-flops would need a 64:1 read mux

7. Clocking and Timing Constraints

- The input clock clk drives the DSP (and the fast-clock registers of M3) through a BUFG. A BUFR divides clk by 6 to produce the system clock used by every other register. No MMCM or PLL is used; an MMCM alone would add about 0.1 W to the total power.
- All crossings between the two clocks are single register-to-register paths. Values from the system clock are re-registered on the fast clock before they reach the DSP, and results are held for six fast cycles before the system clock reads them. Vivado times all crossings because both clocks come from the same source.
- The XDC contains one create_clock on clk; Vivado derives the divided clock from the BUFR. The top-level ports are set as false paths because no board pins are assigned in this phase, so the reported Fmax reflects the design itself.
- The FOM run uses a 10 ns clock (DSP 100 MHz, system 16.7 MHz). The maximum-frequency run uses 3.333 ns (DSP 300 MHz, system 50 MHz), which meets timing with WNS +0.055 ns.

8. Fixed-Point Bit-Width Analysis

Pixels are UQ8.0 (unsigned, 0-255): 8 bits carry the full dynamic range of a grayscale sensor, and any narrower format would lose information before the convolution. Weights are Q3.4 (8-bit signed, 4 fractional bits), which covers the common edge, blur and sharpen kernels (for example Sobel and Laplacian, whose integer weights of up to 4 need 3 integer bits) with a resolution of 1/16. Every intermediate width in Table 6 is the smallest that holds its worst case, computed with all weights at -128 and all pixels at 255.

Table 6. Bit widths along the datapath.

Signal	Format	Bits	Worst-case range	Notes
Pixel q	UQ8.0	8	0 to 255	Unsigned; extended with a 0 to 9 bits signed at the DSP B port
Weight w	Q3.4	8	-128 to 127 (-8.0 to 7.9375)	Signed
Product q x w	Q11.4	16	-32640 to 32385	One DSP lane
Packed operand A	-	25	$w_0 \times 2^{16} + w_1$	Fits the DSP48E1 A port exactly
DSP output P (used)	-	32	two 16-bit lanes	Low lane offset by 2^{15}
Column sums A, B, C	Q13.4	18	3 products: +/-97920	
T2	Q14.4	19	6 products: +/-195840	
M3 result y	Q15.4	20	9 products + 8: +/-293760	Includes the rounding bias
Rounded value	Q16.0	17	-18360 to 18217	$y \gg 4$, sign-extended by one bit
Output pixel	Q15.0	16	-18360 to 18217	Overflow impossible (< 32767); ReLU optional

Rounding is round-half-up, as specified in the golden model: $(\text{sum} + 8) \gg 4$ with an arithmetic shift. Truncation (no +8) would bias every output downward by half a least-significant bit on average. IMG-TC11 (all weights $1/16$) exercises many exact .5 ties.

9. RTL Implementation Details

All RTL is Verilog-2001 and parameterised by PIXEL_WIDTH, WEIGHT_WIDTH, IMG_WIDTH, ACC_WIDTH, OUT_WIDTH, FRAC_BITS and NUM_KERNELS. Table 7 gives the resource usage of each module after implementation.

Table 7. Post-implementation utilization per module (10 ns clock).

Instance	LUTs	of which LUT-RAM	Flip-flops	DSP
u_clk_gen (BUFG + BUFR)	0	0	0	0
u_m1_line_buffer	0	0	512	0
u_m2_window_generator	3	0	10	0
u_m3_mac	135	0	588	1
u_m4_kernel_storage	27	8	28	0
u_m5_output_handling	1	0	34	0
u_m6_control_fsm	18	0	20	0
u_m7_fifo	42	24	41	0
Total	226	32	1233	1

9.1 Implementation techniques

- **Flip-flops instead of LUTs.** Delay lines and state are kept in flip-flops (shreg_extract = "no" prevents Vivado from turning them into LUT-based shift registers).
- **DSP inference.** The multiply-add is written as a registered $A \times B + \text{constant}$ in its own module with use_dsp = "yes", so Vivado absorbs every register into the DSP48E1 (AREG/BREG, MREG, PREG) and the constant into the post-adder.
- **LFSR counters.** Counters that only need to detect a fixed count (pixels per frame, pixels per row, released results) are LFSRs: one feedback LUT plus an equality compare, instead of a carry chain plus a compare. The feedback taps were checked to give maximal-length sequences for widths 3 to 20.
- **No dead logic.** Saturation is built only when overflow is possible, ReLU uses reset pins, the FIFO has no occupancy counter (it equals wr_ptr - rd_ptr), and the rounding constant is preloaded rather than added.
- **Reset policy.** Only control state is reset (valid bits, FSM, pointers, counters). Data registers load only when their valid signal is high and are never read unqualified, which keeps the reset network small.
- **Stall handling.** Every data stage is enabled by its own valid signal, so input stalls freeze the pipeline and cost no extra logic.

10. Verification

10.1 Golden reference model

python/golden_model_image_flow.py loads any image, converts it to grayscale, resizes it to IMG_WIDTH x IMG_WIDTH, and computes the reference output with exactly the hardware arithmetic: an integer 3 x 3 multiply-accumulate, round half up ((sum + 8) >> 4), clamp to 16-bit signed, and optional ReLU. It writes the image, kernels and expected outputs as hex files for all 16 tests, and PNG previews. Test data for the extended tests is seeded, so every run produces identical files.

10.2 Testbenches

Each testbench loads the kernels through the write port, starts a pass, streams the image following the pixel_req handshake, compares every output with the expected hex file and writes the hardware output to a hex file that python/visualize_hw_outputs.py renders back into an image. A test passes when it prints TOTAL ERRORS: 0. IMG-TC7 to IMG-TC16 share a common testbench body and additionally check that all pixels were sent, frame_done rises, no output appears after the frame, and no watchdog timeout occurs.

Table 8. Test cases.

Test	Purpose
IMG-TC1	Basic correctness with the Sobel Gx kernel (also records the SAIF for power analysis)
IMG-TC2	Multi-kernel: Gx in bank 0 and Gy in bank 1, two back-to-back passes
IMG-TC3	ReLU disabled and enabled on the same image
IMG-TC4	Gapless throughput: 900 outputs in 900 consecutive cycles
IMG-TC5	Edge-detection demo: Gx and Gy combined into an edge-magnitude image
IMG-TC6	Input stalls while streaming
IMG-TC7	All weights -128 on a noise image (worst case for the packed lanes)
IMG-TC8	All weights +127 on an all-255 image (largest positive result)

Test	Purpose
IMG-TC9	Mixed -128/127/-1/0/1 weights: every high/low lane sign combination
IMG-TC10	Identity kernel: each output must equal the window centre
IMG-TC11	All weights 1/16: many exact rounding ties
IMG-TC12	Seeded random signed kernel
IMG-TC13	Three back-to-back passes with a kernel reload between passes
IMG-TC14	About 30% random input stalls with a Laplacian kernel
IMG-TC15	Reset in the middle of a pass, then a bit-exact fresh pass
IMG-TC16	Bank switch with ReLU toggled; writes to non-existent taps 9-15 must be ignored

10.3 Test images

test_images/shapes/ contains 14 synthetic 32 x 32 images (arrow, fish, heart, house, key, lightning, moon, rocket, sphere, square, star, tree, triangle, umbrella) with sharp corners, diagonals, curves, thin lines and holes. test_images/photos/ contains six natural photographs that the golden model resizes to the chosen width. The whole flow (golden model, all 16 simulations, rendering) runs with one command: run_image_flow.bat <image> [IMG_WIDTH].

10.4 Results

All 16 image testbenches produce outputs identical to the golden model (TOTAL ERRORS: 0) for the 32 x 32 test images. The rendered hardware images in sim/image outputs/ are pixel-identical to the golden previews.

Waveform capture flow

The captures below were taken interactively rather than through the batch run_image_flow.bat flow, which runs vsim in headless (-c) mode and closes automatically once a test finishes. For each test shown, the golden model was generated once for the chosen image (python python/golden_model_image_flow.py test_images/shapes/sphere.png --img-width 32), the RTL and testbenches were compiled into a work library (vlib work; vlog +incdir+../testbenches ../rtl/*.v ../testbenches/*.v), and the target testbench was then launched without -c so the ModelSim/Questa GUI stayed open (vsim -voptargs=+acc -gIMG_WIDTH=32 work.<testbench_name>). Inside the GUI, add wave * added the testbench's top-level signals, run -all ran the test to completion, and the Wave window was then zoomed to the region of interest for each screenshot below.

Four of the sixteen image tests are illustrated: IMG-TC1 (basic correctness), IMG-TC2 (multi-kernel bank switching), IMG-TC4 (gapless throughput) and IMG-TC6 (input stall handling). All sixteen produce TOTAL ERRORS: 0, per Table 8 and the summary above.

IMG-TC1 — Basic Correctness

IMG-TC1 streams a 32 x 32 image through the accelerator with the Sobel Gx kernel and checks every output against the golden model, while also recording the SAIF used for the power estimate in Section 11.3. The three captures below follow one pass from kernel load through to completion.

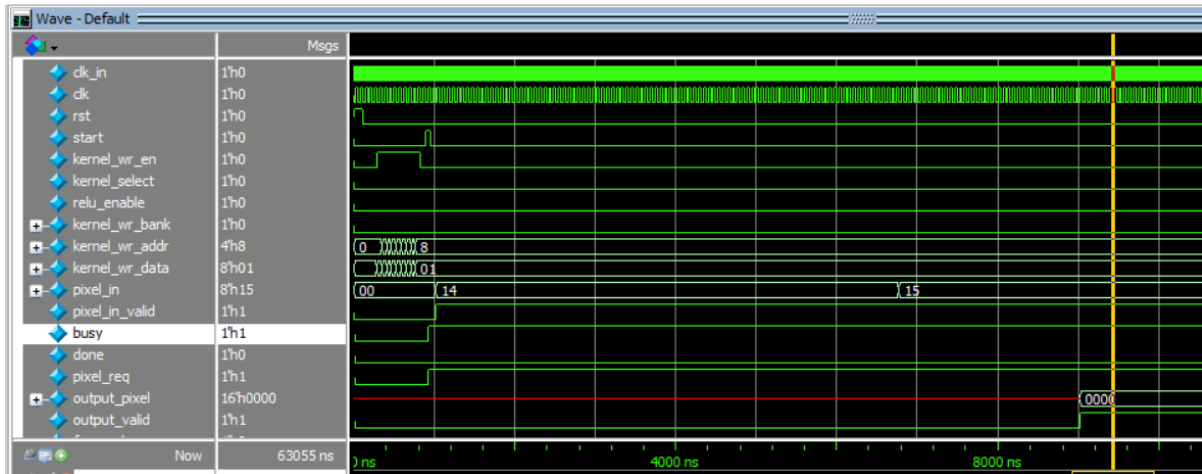


Figure 5. IMG-TC1: the Gx kernel loading into bank 0 (*kernel_wr_en* / *kernel_wr_addr* / *kernel_wr_data*), the start pulse, and *pixel_in* / *pixel_in_valid* streaming beginning; *busy* and *pixel_req* go high and the first *output_valid* pulse appears with *output_pixel* = 0x0000.

This is the setup phase: the nine Gx taps are written into bank 0, start launches the pass, and the FSM's STREAM state begins requesting pixels. *output_valid* rises for the first time here, once the line buffer and MAC pipeline have filled enough to produce the first valid 3 x 3 window's result.

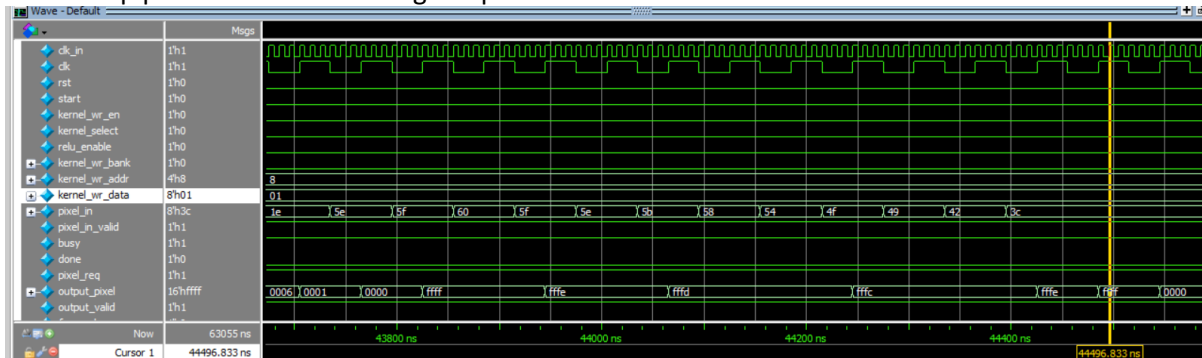


Figure 6. IMG-TC1: a mid-stream slice ($\approx 43.8\text{--}44.5\ \mu\text{s}$) with *pixel_in* advancing every cycle while *output_valid* stays high and *output_pixel* cycles through a run of distinct results (0x0006, 0x0001, 0x0000, 0xFFFF, 0xFFFE, 0xFFFD, 0xFFFC, 0xFFFE, 0xFFFD, 0xFFFE, 0x0000).

This slice is well past the pipeline fill, so it shows steady-state operation: every system-clock cycle advances *pixel_in*, and *output_valid* stays asserted with *output_pixel* taking on a distinct 16-bit signed value each cycle rather than repeating or freezing — visual evidence that the MAC pipeline and output stage are producing genuine, varying Sobel-Gx results.

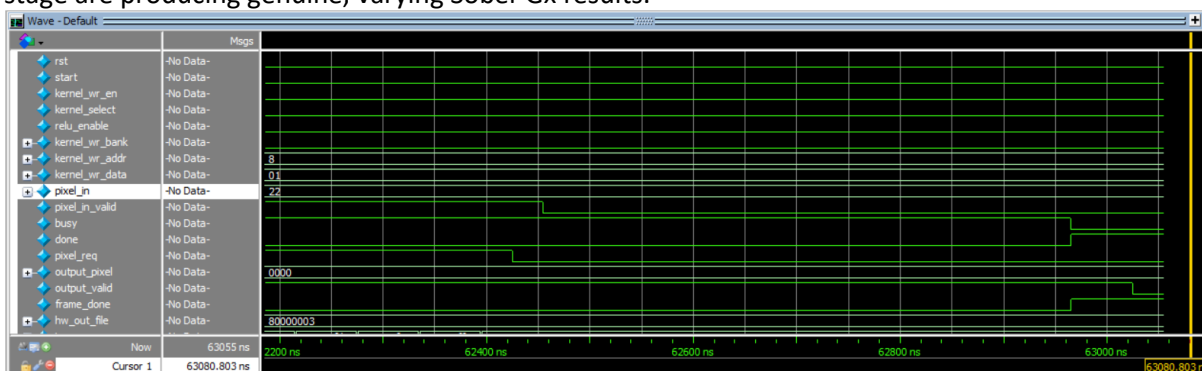


Figure 7. IMG-TC1: end of the pass — *pixel_in_valid* and *pixel_req* fall as the last pixel is accepted, *busy* drops, and *done* / *frame_done* rise together, marking a clean completion.

This is the tail of the run: once the 1024th pixel has been accepted, *pixel_req* and *pixel_in_valid* drop, *busy* deasserts, and the FSM's *done* / *frame_done* outputs rise to signal the pass is finished. Correctness itself is not visible in a waveform — it is verified numerically by the testbench comparing

every output_pixel against the golden model, which reports TOTAL ERRORS: 0 in the simulation transcript for this test.

IMG-TC2 — Multi-Kernel Selection

IMG-TC2 loads both kernel banks up front — bank 0 with the Sobel Gx taps, bank 1 with Gy — then runs two full 32 x 32 passes back-to-back, selecting kernel_select = 0 for the first pass and kernel_select = 1 for the second, each checked against its own golden output.

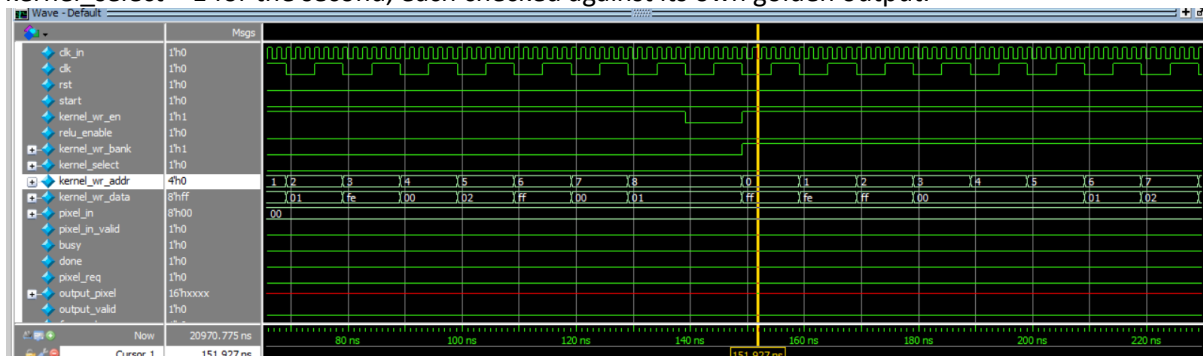


Figure 8. IMG-TC2: kernel_wr_en pulses twice in sequence, with kernel_wr_bank held low (bank 0, Gx taps at addresses 0-8) and then high (bank 1, Gy taps), loading both banks before either pass starts.

Because both kernel banks are written up front, this capture shows the setup phase for the whole test: nine writes into bank 0 followed immediately by nine into bank 1, each with its own kernel_wr_data value. No pixels are streamed yet (pixel_in_valid stays low) — pass 1 only begins once both banks are loaded.

```
#          Using alternate file: ./wlfti423zx
# --- IMG-TC2 PASS 1: kernel_select=0 ---
# PASS 1: all 1024 pixels sent
# PASS 1: frame_done asserted
# --- IMG-TC2 PASS 2: kernel_select=1 ---
# PASS 2: all 1024 pixels sent
# PASS 2: frame_done asserted
#
```

Figure 9. IMG-TC2 transcript: Pass 1 (kernel_select = 0, Gx) completes with all 1024 pixels sent and frame_done asserted, followed immediately by Pass 2 (kernel_select = 1, Gy) completing the same way.

This is the direct evidence that the bank switch works: Pass 2 launches with a fresh start pulse right after Pass 1's frame_done, now reading from bank 1 instead of bank 0, and both passes finish cleanly. The full transcript (not reproduced here) ends with IMG-TC2 MULTI-KERNEL: TOTAL ERRORS: 0, confirming every output in both passes matched its respective Gx / Gy golden model.

IMG-TC4 — Gapless Throughput

IMG-TC4 re-runs the same correctness check as IMG-TC1, but additionally tracks output_valid every cycle from the first output onward: once release begins, any single low cycle before all 900 outputs are produced is logged as a gap and fails the test. This is the direct evidence for the one-output-pixel-per-cycle throughput claim in Table 1.

```

Release began at output #0
PASS: all 1024 pixels sent
PASS: frame_done asserted
PASS: zero gaps -- sustained 1 pixel/cycle confirmed across all 900 outputs
-----
IMG-TC4 GAPLESS THROUGHPUT: 900/900 outputs checked, gaps=0, TOTAL ERRORS: 0
-----
** Note: $stop      : ../testbenches/tb_imgtc4_gapless_throughput.v(104)
   Time: 10504963 ps Iteration: 2 Instance: /tb_imgtc4_gapless_throughput

```

Figure 10. IMG-TC4 transcript: release begins at output #0, all 1024 pixels are sent, frame_done asserts, and the test reports zero gaps across all 900 outputs.

The “zero gaps” line is the test’s own pass/fail criterion for throughput, separate from output correctness — a design that computed every value correctly but stalled even once between outputs would fail this specific check. TOTAL ERRORS: 0 alongside it confirms the 900 outputs were also numerically correct.

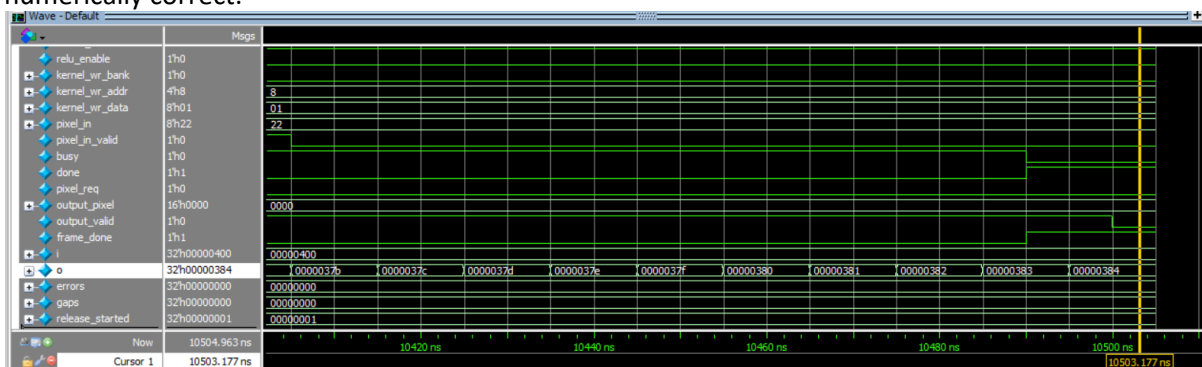


Figure 11. IMG-TC4: internal testbench counters just before completion — o = 0x384 (900, the full output count) with errors = 0 and gaps = 0, moments before frame_done rises.

Because these are the testbench’s own bookkeeping signals (i, o, errors, gaps, release_started), this capture is a direct look at the pass/fail state the transcript above is summarising, rather than the DUT’s own ports — errors and gaps both sitting at zero right as the last output is produced is waveform-level confirmation of the transcript’s result.

IMG-TC6 — Stall Handling

IMG-TC6 checks that the accelerator tolerates a pixel source that is not always ready. Twice during the run, pixel_in_valid is withheld for 6 consecutive cycles while pixel_req stays high — once early (pixel #51, ~5% in) and once at the midpoint (pixel #512) — and the pass must still complete with a bit-exact result and no dropped or duplicated pixels.

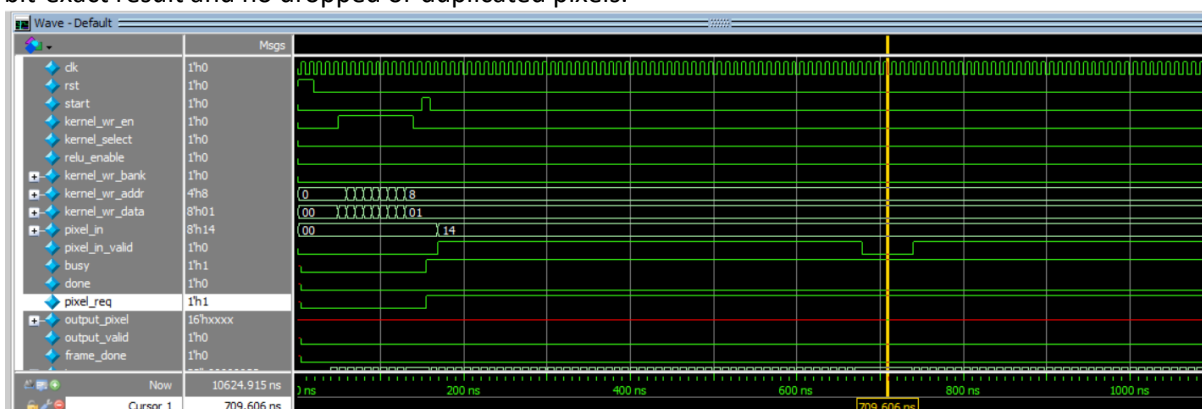


Figure 12. IMG-TC6, stall #1 ($\approx t = 650\text{--}710$ ns): pixel_in_valid dips low for 6 cycles while pixel_req stays high and busy remains asserted throughout.

This is the request/valid handshake under stress: the accelerator keeps asking for data (`pixel_req = 1`) the whole time, but the source simply doesn't provide it for six cycles. `busy` never drops, so the FSM is paused, not reset or restarted, and resumes the moment `pixel_in_valid` returns.

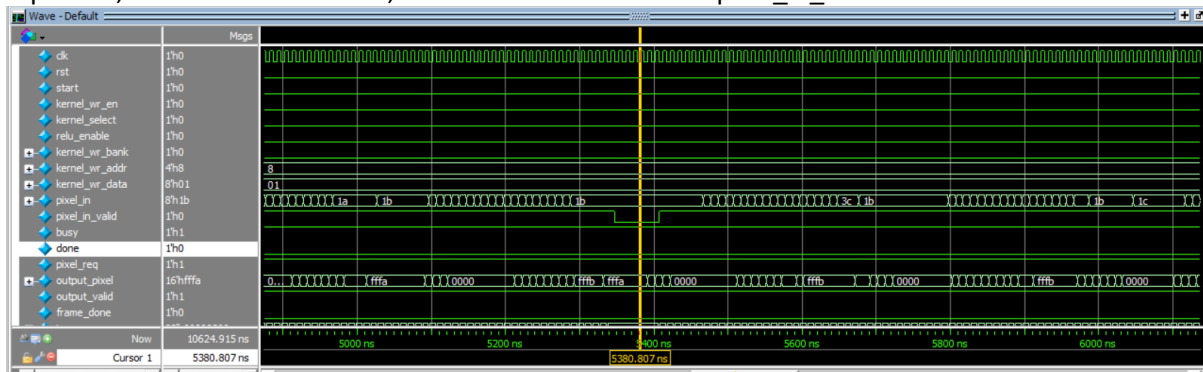


Figure 13. IMG-TC6, stall #2 ($\approx t = 5300\text{--}5400\text{ ns}$): the same 6-cycle withholding of `pixel_in_valid` partway through the pass, with `pixel_in` and `output_pixel` continuing to advance normally immediately before and after. The second stall lands mid-stream rather than near the start, confirming the same pause/resume behaviour holds once the pipeline is already full of in-flight data, not just during the initial fill.

```
Using alternate file: ./wlftrfwg2i
--- stall #1: withholding pixel_in_valid for 6 cycles at pixel #51 ---
--- stall #2: withholding pixel_in_valid for 6 cycles at pixel #512 ---
PASS: all 1024 pixels sent despite 12 stall cycles
PASS: frame_done asserted

=====
IMG-TC6 STALL HANDLING: 900/900 outputs checked, stall cycles=12, TOTAL ERRORS: 0
=====
```

Figure 14. IMG-TC6 transcript: both stalls are logged at the expected pixel indices (#51 and #512), all 1024 pixels are still sent despite the 12 injected stall cycles, and the test reports **TOTAL ERRORS: 0**.

This is the numerical confirmation behind the two waveform captures above: despite twelve cycles where no new pixel was accepted, every one of the 900 outputs still matched the golden model exactly, and no pixel was lost or duplicated across either stall.

11. FPGA Implementation Results

Synthesis and implementation used Vivado 2018.2 for the xc7z020clg400-1 device with default strategies. Power was estimated by Vivado's `report_power` with switching activity from a SAIF file recorded in simulation during the active part of IMG-TC1 (gated on `busy`), at the same clock period as the implementation run, giving confidence level High.

11.1 Utilization

Name	Slice LUTs (53200)	Slice Registers (106400)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	LUT Flip Flop Pairs (53200)	DSP s (220)	Bonded IOB (125)	BUFGCTRL (32)	BUF R (16)
cnr_accelerator_top	226	1233	240	194	32	172	1	49	1	1
u_clk_gen (clk_gen_6x)	0	0	0	0	0	0	0	0	1	1
u_m1_line_buffer (line...	0	512	81	0	0	0	0	0	0	0
> u_m2_window_gener...	3	10	4	3	0	3	0	0	0	0
> u_m3_mac (M3)	135	588	138	135	0	117	1	0	0	0
u_m4_kernel_storage ...	27	28	15	19	8	10	0	0	0	0
u_m5_output_handlin...	1	34	8	1	0	0	0	0	0	0
> u_m6_control_fsm (co...	18	20	10	18	0	5	0	0	0	0
> u_m7_fifo (FIFO)	42	41	16	18	24	29	0	0	0	0

Figure 15. Utilization by module, 10 ns clock (Vivado).

Name	Slice LUTs (53200)	Slice Registers (106400)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	LUT Flip Flop Pairs (53200)	DSP s (220)	Bonded IOB (125)	BUFGCTRL (32)	BUFR (16)
▼ cnv_accelerator_top	229	1233	249	197	32	178	1	49	1	1
u_clk_gen (clk_gen_6x)	0	0	0	0	0	0	0	0	1	1
u_m1_line_buffer (line...	0	512	91	0	0	0	0	0	0	0
> u_m2_window_gener...	3	10	4	3	0	3	0	0	0	0
> u_m3_mac (M3)	138	588	129	138	0	121	1	0	0	0
u_m4_kernel_storage ...	27	28	13	19	8	10	0	0	0	0
u_m5_output_handlin...	1	34	12	1	0	0	0	0	0	0
> u_m6_control_fsm (co...	18	20	13	18	0	4	0	0	0	0
> u_m7_fifo (FIFO)	42	41	16	18	24	29	0	0	0	0

Figure 16. Utilization by module, 3.333 ns clock (Vivado).

11.2 Timing

Design Timing Summary		
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.923 ns	Worst Hold Slack (WHS): 0.059 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 2477	Total Number of Endpoints: 2477	Total Number of Endpoints: 1268
All user specified timing constraints are met.		

Figure 17. Timing summary, 10 ns clock: WNS +5.923 ns, WHS +0.059 ns.

Design Timing Summary		
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.055 ns	Worst Hold Slack (WHS): 0.075 ns	Worst Pulse Width Slack (WPWS): 1.166 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 2477	Total Number of Endpoints: 2477	Total Number of Endpoints: 1268
All user specified timing constraints are met.		

Figure 18. Timing summary, 3.333 ns clock (300 MHz): WNS +0.055 ns, WHS +0.075 ns; all constraints met.

11.3 Power

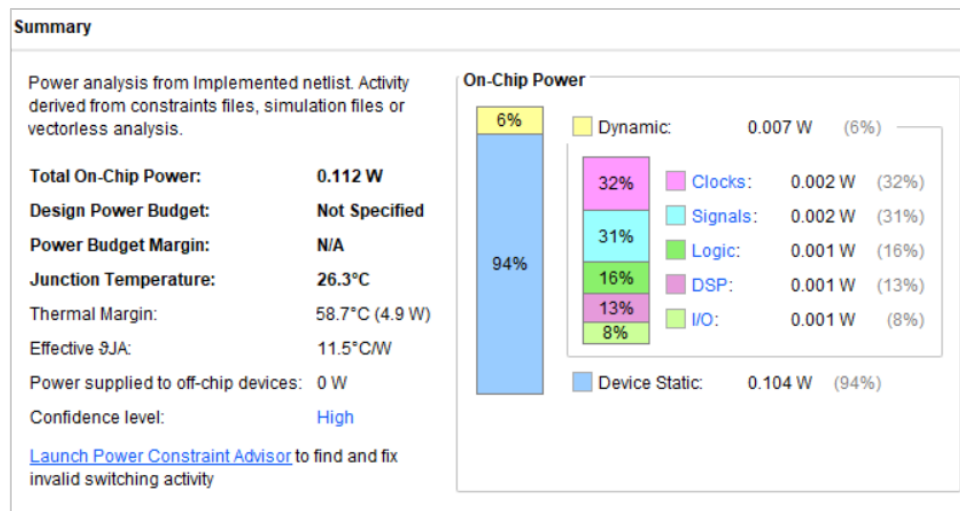


Figure 19. Power, 10 ns clock: 0.112 W total (static 0.104 W, dynamic 0.007 W), confidence High.

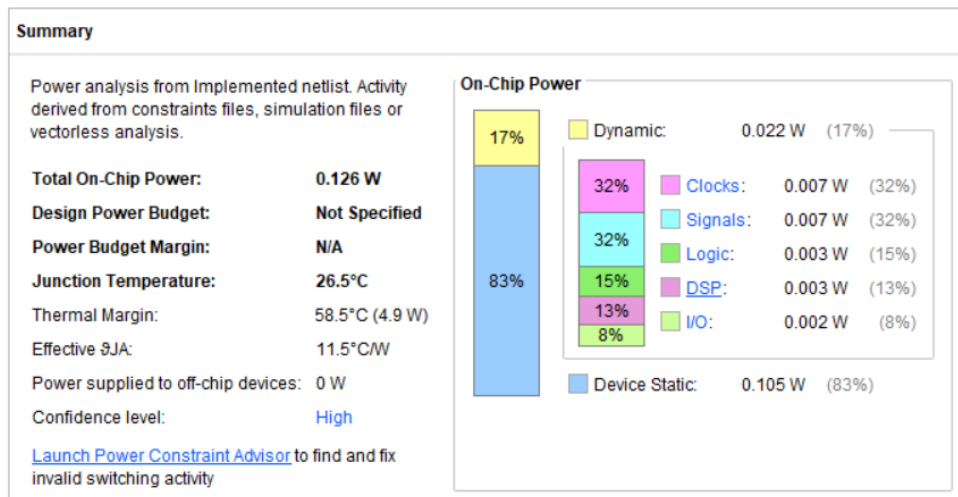


Figure 20. Power, 3.333 ns clock: 0.126 W total (static 0.105 W, dynamic 0.022 W), confidence High.

Device static power (0.104 W) is the leakage of the whole xc7z020 die and does not depend on the design; it is about 93% of the total at the reference clock. Design choices therefore mainly act on the area term of the FOM.

11.4 Placement

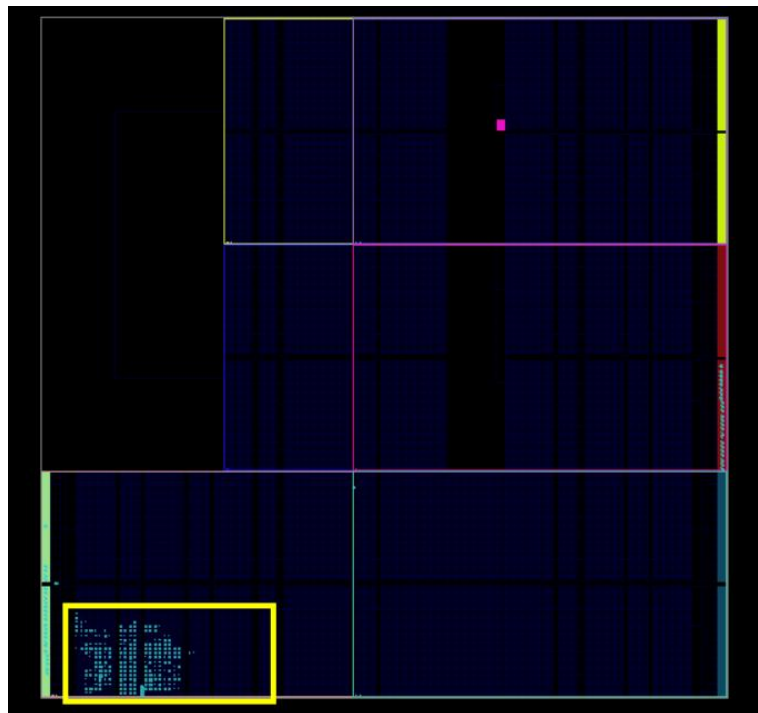


Figure 21. Placed design on the device: the whole accelerator fits in clock region X0Y0, as required by the BUFR.

11.5 Figure of Merit

$$\text{FOM} = \text{Throughput} / (\text{Power} \times (\text{LUTs} + 50 \times \text{DSPs} + 100 \times \text{BRAMs}))$$

Table 9. FOM calculation.

Quantity	10 ns clock (reference)	3.333 ns clock (Fmax)
Throughput (output pixels / system cycle)	1	1
Total power (W)	0.112	0.126
LUTs + 50 x DSPs + 100 x BRAMs	226 + 50 + 0 = 276	229 + 50 + 0 = 279
FOM	1 / (0.112 x 276) = 0.0324	1 / (0.126 x 279) = 0.0284

12. Design Trade-offs and Optimisation History

The design evolved from a conventional 9-multiplier adder tree to the final single-DSP engine. Each step was evaluated against the FOM (Table 10).

Table 10. Design evolution (total power, throughput 1).

Step	LUTs	DSP	FOM	Main change
Original design	316	9	0.0107	Nine multipliers and a 4-level adder tree
Transposed form, packed lanes	367	6	0.0124	Two products per DSP; 9 to 6 DSPs
6x multi-pumped, no MMCM	408	1	0.0178	All operations on one DSP; BUFG + BUFR clocking
LUT-lean datapath	-	1	0.0232	Operand ring, fast-clock accumulation, serial kernel stream, FF line buffer
LFSR FIFO and FSM	230	1	0.0271	No occupancy counter; LFSR counters
Final	226	1	0.0324	M5/M2 simplification, paused kernel stream, SAIF at the implemented clock

12.1 Decisions and alternatives

- **One multi-pumped DSP instead of several DSPs.** One DSP costs 50 LUT-equivalents; the six-fold clock and the operand ring cost only flip-flops and a few LUTs. The price is that the system clock is one sixth of the DSP clock (50 MHz at 300 MHz).
- **No MMCM.** A 6x clock from an MMCM would allow a faster system clock for the same input, but the MMCM alone adds about 0.1 W, roughly doubling total power. Dividing the input clock with a BUFR costs almost nothing.
- **Throughput kept at 1.** Producing more outputs per cycle (wider input or parallel kernels) grows area and dynamic power almost as fast as throughput. Time-sharing the DSP further lowers throughput faster than it saves area.
- **Kernel size 3 x 3.** A larger kernel increases the work per output without increasing throughput. A 2 x 2 kernel would score higher, but 3 x 3 is the standard CNN kernel and is required for the Sobel edge-detection demo.
- **Weights must stay programmable.** Constant-coefficient techniques (shift-and-add, CSD) are not applicable.

13. Bonus Features

Table 11. *Bonus items from the announcement.*

Bonus item	Status	Evidence
One-output-pixel-per-cycle pipelined architecture	Implemented	Gapless output, IMG-TC4
Multiple kernels	Implemented (2 banks)	IMG-TC2, IMG-TC13, IMG-TC16
ReLU activation	Implemented	IMG-TC3, IMG-TC16
Edge-detection demo	Implemented	IMG-TC5: Sobel Gx + Gy edge magnitude
Board demonstration	Planned	The 125 MHz PYNQ-Z2 clock can drive the design directly (system clock 20.8 MHz)

14. Assumptions

1. Throughput is the steady-state number of output pixels per system clock cycle once the output stream has started (1, gapless).
2. Power is the total on-chip power reported by Vivado report_power, including device static power, with switching activity from a SAIF recorded at the same clock period as the implementation run.
3. The reference FOM uses a 10 ns clock on clk the maximum-frequency result uses 3.333 ns. Both are reported.
4. The image is square, IMG_WIDTH = 32, and the convolution is "valid" (no padding), giving 30 x 30 outputs.
5. Kernel banks are written before start and not changed during a pass; banks are not cleared by reset.
6. The pixel source delivers pixels in raster order whenever pixel_req is high; it may stall at any time.

15. Conclusion

The accelerator meets every mandatory specification and implements four of the five bonus items. By combining a transposed-form convolution, two products per DSP operation and six-fold multi-pumping, all nine multiplications of a 3 x 3 window run on one DSP48E1, and the rest of the design was rebuilt so that storage and control use flip-flops and LUT-RAM rather than counted logic. The result is 226 LUTs, 1 DSP and no BRAM for one output pixel per cycle, meeting timing at 300 MHz, with a Figure of Merit of 0.0324 at the reference clock.

Appendix: reproducing the results

- Golden model, simulations and rendering: run_image_flow.bat
test_images\shapes\sphere.png
- Simulations only: vsim -c -do "do sim/scripts/run_all_image_tests.do 32